Telegram to Notesnook Bridge: Deployment Guide (v1.4)

Oct 18, 2026

What you'll build

You'll run a small private Telegram bot that saves every message you send it as a note in your Notesnook account, formatting, checklists, photos and text files included. The bot runs on Fly.io, so it keeps working when your own computer is off.

It can also tag your notes from Telegram with `#tags`, and it sends you a backup of your tag list after every change, so a lost disk never costs you your tags. Tags are optional; skip them and the rest works the same.

Beyond plain messages, the bot has commands for saving raw HTML (`/html`), saving callout boxes (`/callout`) and choosing how todos are saved (`/todostyle`).

1. You send a message to your bot in Telegram.
2. The bot, running on Fly.io, converts the message to HTML.
3. It posts the note to Notesnook's Inbox API using your API key. With a self-hosted server, it first encrypts the note with your account's public key.
4. The note appears in Notesnook after your next sync, on every device.

The whole setup takes about 30 minutes. You need:

- A **Telegram** account.
- A **Notesnook** account with the Inbox API available. It was added in Notesnook 3.4 and is still labeled beta, so check Settings > Inbox in your app.
- A **Fly.io** account. Fly asks for a payment card, and the bot runs on its smallest machine. Check Fly.io pricing for current rates.
- A computer with a terminal. You don't need to install Python locally; Fly builds the bot for you.

The bot only answers you. Anyone else who finds it is ignored.

What's new in v1.4.0

Version 1.4.0 fixes the checklist header count, adds three commands (`/html`, `/callout` and `/todostyle`) and adds syntax highlighting for embedded code files. It needs no new secrets and no new dependencies. `requirements.txt`, the `Dockerfile` and `fly.toml` are unchanged.

Fixes

- **Checklist header count.** Todos are now saved with the classes Notesnook itself uses (`checklist--item`, plus `checked` when done). Before, the checklist header could show `0/2` even when an item was checked, because the bot left off the `checklist--item` class that the header counts.
- **`/help` and `/setup` too long for Telegram.** Telegram rejects messages over 4,096 characters, and `/help` had grown past that. Both commands now split at blank lines into several messages when needed, with no text lost.

New commands

- **`/html`** saves the HTML after the command exactly as typed. Normal messages escape any tags you type, so use this when you want real HTML.
  - The note title is the first heading (`<h1>` to `<h6>`), or the date and time if there is none.
  - A `#tag` on the last line works as usual.
  - Every tag must be closed. The bot refuses to save HTML with an unclosed or mismatched tag, because Notesnook would turn the whole note into a code block.
  - Example: `/html <h1>Groceries</h1><ul class="checklist"><li class="checklist--item">milk</li></ul>`
- **`/callout`** saves a Notesnook callout box.
  - One-message form: `/callout warning Back up first`, then the body on the next lines.
  - Sent on its own, the bot asks for the type, the title and the text, one message each. Send `skip` to leave a step out and `/cancel` to stop. An unanswered prompt expires after 5 minutes.
  - Types: note, abstract, summary, tldr, info, todo, tip, hint, important, success, check, done, question, help, faq, warning, warn, caution, attention, failure, fail, missing, danger, error, bug, example, quote, cite. No type, or an unknown one, gives `info`. No title uses the type name in capitals.
  - The body can use lists, todos and Telegram formatting. The whole message becomes the callout, and the note title is the callout title.
  - Tags work at the end of the title line, at the end of the last line, or on a line of their own. Saved tags are applied and removed from the text. An unknown tag stays in the text and the reply says it was not found.
  - A title that starts with a type word is read as the type. To keep the word in the title, give a type first: `/callout info Tip of the day`.
- **`/todostyle`** switches how todos are saved. It is remembered across restarts.
  - `header` (default): Notesnook's task list, with a title bar, a progress bar and a `checked/total` count. The line above the todos becomes the title.

- `simple`: a plain checklist with no header or count. The line above the todos stays in the note as normal text.
- Send `/todostyle` to switch, or `/todostyle header` or `/todostyle simple` to set one directly. `/ver` shows the current style.
- The style applies to todos in normal messages and in `/callout` bodies. It does not change `/html`, which sends your markup as typed.
- The setting is stored in the tag file, so it survives redeploys when a Fly volume is mounted. The tag backup file does not include it, so set the style again after a `/restore`.

Improvements

- **Syntax highlighting for embedded text files.** Files sent to the bot now get a code language by extension: `.py`, `.js`, `.ts`, `.json`, `.yaml` and `.yml`, `.sh`, `.sql`, `.md` and `.markdown`, `.css`, `.toml`, `.ini`, `.csv`, and `.html` and `.xml` (saved as Notesnook's `markup`). Other file types stay plain text.
- **Help and menus.** `/help` has new HTML, CALLOUTS and TODO STYLE sections, and the photos-and-files section mentions highlighting. `/html`, `/callout` and `/todostyle` appear in the Telegram command menu. `/cancel` also stops a `/callout` prompt.
- **Version.** The bot and the `/setup` text now report 1.4.0.
- **Guide catch-up.** This guide now also covers features from 1.3.1 that it had been missing: `/tag` and `/untag` ask for any missing details when sent on their own, and `/cancel` stops one of those prompts.

Upgrading from 1.3

1. Replace `bot.py` with the 1.4.0 version.
2. Run `fly deploy`. No new secrets or dependencies are needed.
3. Send `/ver` and check that it reports `v1.4.0` and `Todo style: header`.
4. Test with a todo list, for example:

   ```
   Groceries:
   - [ ] milk
   - [x] eggs
   ```

   The checklist header should read `1/2`. Notes saved before the upgrade keep their old count.

Good to know

- **A tag shows as applied, but the note has no tag.** The bot's "tags: name" reply only means it sent a saved tag ID. Notesnook silently skips any tag ID that does not exist in the account it syncs with. Tag IDs are per account, so if you switch between default and self-hosted Notesnook, copy each tag's ID again (right-click the tag, then Copy ID) and re-save it with `/tag name id`.

- **Extra tags in Telegram's `#` list.** The suggestions come from Telegram itself, which offers hashtags you have used in your chats. They do not come from the bot. `/tags` lists the tags the bot knows.

Default or self-hosted Notesnook?

Version 1.2 can send notes to either backend. One setting decides which: `NOTESNOOK_SERVER_URL`. Leave it unset for default Notesnook, or set it to your own server's address.

	Default Notesnook	Self-hosted server
Use it if	You sync with Notesnook's own servers	You run your own Notesnook sync server
`NOTESNOOK_SERVER_URL`	Not set	Your server address, such as `https://notesnook.example.com`
Where to create the Inbox key	Your default Notesnook account	Your self-hosted account
Who encrypts the note	Notesnook's inbox server	The bot, with your account's public key
Extra setup	None	Set the server address, and use a key from the self-hosted account
Check with `/ver`	Destination: hosted Notesnook inbox	Destination: your server address

The other steps are the same for both. Where a step differs, it is marked **Self-hosted**.

Step 1: Create the Telegram bot

You need two values from Telegram: a bot token and your own numeric user ID.

1. In Telegram, open a chat with **@BotFather**.
2. Send `/newbot`. Pick a display name, then a username that ends in `bot`.
3. BotFather replies with a token that looks like `123456789:ABCdef...`. Copy it. Anyone with this token controls your bot, so treat it like a password.
4. Open a chat with **@userinfobot** and copy the number it replies with. This is your user ID. The bot will only respond to this ID.

Keep both values handy. You'll store them as secrets in Step 4.

Step 2: Turn on the Notesnook Inbox API

The Inbox API is a private mailbox for your account. Other services post notes to it with a key, and the notes show up after your next sync, encrypted so only you can read them.

These steps are for default Notesnook. If you run your own server, create the key on your self-hosted account instead; the self-hosted steps follow this list.

1. In Notesnook, open **Settings > Account > Inbox**. On some versions it is **Settings > Inbox**.
2. Turn on **Enable Inbox API**. Unless you have your own PGP key pair, let Notesnook generate the keys.
3. Under API Keys, click **Create Key**. Name it something like `telegram` and choose an expiry.
4. Copy the key and keep it private.

The default expiry is one month. When a key expires, the bot starts replying with a 401 error. Choose a longer expiry or Never if you don't want to rotate it, and see the troubleshooting section for how to replace a key.

The Inbox API is still in beta, so details may change. The official guide has the current request format.

Self-hosted: create the key on your own account

The server only accepts notes encrypted for your account, so the key has to come from the same account that the server holds. The bot looks up your account's public encryption key on its own, using the Inbox key, and encrypts each note with it. You never paste a separate encryption key.

1. Open the Notesnook app that is connected to your own server. If it is still pointing at default Notesnook, log out first: the server address can only be changed while logged out. Then log in to your self-hosted account.
2. Open **Settings > Account > Inbox** and turn on **Enable Inbox API**.
3. Under API Keys, click **Create Key**, name it `telegram`, choose an expiry, and copy the key.
4. Keep the key for Step 4. You will store it as the `NOTESNOOK_INBOX_KEY` secret, together with your server address in `NOTESNOOK_SERVER_URL`.

A key from a default Notesnook account does not work on your server. If the bot replies "Failed (401)" after you switch, you are using the wrong account's key. Create a new key on the self-hosted account.

Use the main sync address for the server, such as `https://notesnook.example.com`, with no slash at the end. Do not use the auth, events, monograph or attachments addresses.

Step 3: Set up the project files

Create a new empty folder, for example `notesnook-bot`, and save these five files in it. Use a dedicated folder: Fly uploads everything in the folder you deploy from, so a crowded folder makes deploys slow.

bot.py

```
"""Telegram -> Notesnook Inbox API bridge.

Send a message to your Telegram bot and it shows up as a note in Notesnook
after your next sync. Bold, italic, links, code and quotes are kept.

The Inbox API has no attachment field, so media is embedded in the note's
HTML, the way the Notesnook email worker does it: photos and image files
become base64 data: URIs and text files become code blocks. Videos, voice
notes and other files get a placeholder line (plus the caption, if any).

Env vars:
  TELEGRAM_BOT_TOKEN   token from @BotFather
  NOTESNOOK_INBOX_KEY  key from Notesnook > Settings > Account > Inbox
  NOTESNOOK_SERVER_URL optional, self-hosted sync server, e.g.
                       https://notesnook.example.com (unset = hosted inbox)
  ALLOWED_USER_ID      your numeric Telegram user ID (only you can use the bot)
  BOT_TIMEZONE         optional, default America/New_York (generic note titles)
  EMBED_MEDIA          optional, set to 0 to turn off embedding (placeholders only)
  TAGS_FILE            optional, where saved tags live (default /data/tags.json
                       when a Fly volume is mounted at /data)
  NOTESNOOK_TAGS       optional, JSON of tag names to IDs used once to seed the file,
                       e.g. {"groceries": "67aecf3b9e1398484554bc90"}
"""

import asyncio
import base64
import html
import io
import json
import os
import re
import subprocess
import tempfile
import threading
import time
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

import requests
from telegram import BotCommand, Message, Update
from telegram.ext import (
    Application,
    CommandHandler,
    ContextTypes,
    MessageHandler,
    filters,
)

HOSTED_INBOX_URL = "https://inbox.notesnook.com/"
# Self-hosted sync server address, e.g. https://notesnook.example.com
# When set, notes are PGP-encrypted here and sent to <server>/inbox/items.
# When empty, the bot uses the hosted Notesnook inbox as before.
SERVER_URL = os.environ.get("NOTESNOOK_SERVER_URL", "").strip().rstrip("/")
BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
INBOX_KEY = os.environ["NOTESNOOK_INBOX_KEY"]
ALLOWED_USER_ID = int(os.environ["ALLOWED_USER_ID"])
TIMEZONE = os.environ.get("BOT_TIMEZONE", "America/New_York")  # for generic titles
__version__ = "1.4.0"  # bump when you change the bot; shown by /ver

ATTACHMENT_SIZE_LIMIT = 5 * 1024 * 1024  # per file, same as the email worker
NOTE_SIZE_LIMIT = 8_000_000  # bytes of HTML per note (the Inbox API allows 10 MB)
EMBED_MEDIA = os.environ.get("EMBED_MEDIA", "1").strip().lower() not in (
```

```
    "0",
    "false",
    "no",
    "off",
)

_IMAGE_MIMES = {"image/jpeg", "image/png", "image/gif", "image/webp"}
_TEXT_MIMES = {
    "application/json",
    "application/xml",
    "application/x-yaml",
    "application/yaml",
    "application/toml",
    "application/javascript",
    "application/x-sh",
}
_TEXT_EXTS = {
    ".txt", ".md", ".markdown", ".csv", ".tsv", ".json", ".xml", ".yaml", ".yml",
    ".toml", ".ini", ".log", ".py", ".js", ".ts", ".html", ".css", ".sh", ".sql",
}
# File extension -> Notesnook code block language (names from Notesnook's own
# language list). Extensions not listed stay plain text.
_CODE_LANGUAGES = {
    ".py": "python",
    ".js": "javascript",
    ".ts": "typescript",
    ".json": "json",
    ".yaml": "yaml",
    ".yml": "yaml",
    ".sh": "bash",
    ".sql": "sql",
    ".md": "markdown",
    ".markdown": "markdown",
    ".css": "css",
    ".toml": "toml",
    ".ini": "ini",
    ".csv": "csv",
    ".html": "markup",  # Notesnook's name for HTML/XML
    ".xml": "markup",
}

# Telegram-only tags Notesnook doesn't know; drop the tag, keep the text.
_TG_ONLY_TAGS = re.compile(r"</?tg-(?:spoiler|emoji)[^>]*>")
_PRE_BLOCK = re.compile(r"(<pre>.*?</pre>)", re.DOTALL)
_BLOCK_TAGS = ("<pre>", "<blockquote")


def media_label(msg: Message) -> str | None:
    """Return a human label if the message carries media, else None."""
    if msg.photo:
        return "Photo"
    for attr, label in (
        ("video", "Video"),
        ("video_note", "Video message"),
        ("animation", "GIF"),
        ("voice", "Voice note"),
        ("audio", "Audio"),
        ("document", "File"),
        ("sticker", "Sticker"),
    ):
        if getattr(msg, attr, None):
            return label
    return None
```

```
_TAG = re.compile(r"<(/?)([a-z][a-z0-9-]*)[^>]*>")
_TASK = re.compile(r"^[-*•]\s+\[([ xX]?)\]\s+(.+)$")
_BULLET = re.compile(r"^[-*•]\s+(.+)$")
_ORDERED = re.compile(r"^\d+[.)]\s+(.+)$")
_HEADING = re.compile(r"^(#{1,6})\s+(.+)$")
_LIST_START = re.compile(r"^(?:[-*•]\s+|\d+[.)]\s+)\S")
_HEADING_MARK = re.compile(r"^#{1,6}\s+")


def _balanced(fragment: str) -> bool:
    """True if every tag opened in the fragment is closed inside it."""
    stack = []
    for m in _TAG.finditer(fragment):
        closing, name = m.group(1), m.group(2)
        if closing:
            if not stack or stack.pop() != name:
                return False
        else:
            stack.append(name)
    return not stack


def _plain_title(line_html: str) -> str:
    """Plain-text version of a line of Telegram HTML, minus a trailing colon."""
    return html.unescape(_TAG.sub("", line_html)).strip().rstrip(":").strip()


def _title_attr(line_html: str | None) -> str:
    """data-title attribute: Notesnook shows it as a checklist's header title."""
    plain = _plain_title(line_html) if line_html else ""
    return f' data-title="{html.escape(plain[:80], quote=True)}"' if plain else ""


def _structured_html(tg_html: str) -> str | None:
    """Convert markdown-style lines (lists, todos, headings) to real HTML.

    Returns None when the message has no such lines, or when it isn't safe to
    split it into lines (formatting that spans lines, code blocks, quotes);
    the caller then falls back to the simple <br> conversion.
    """
    if "<pre" in tg_html or "<blockquote" in tg_html:
        return None
    lines = tg_html.split("\n")
    if not all(_balanced(line) for line in lines):
        return None

    parsed = []  # (kind, text, extra)
    for line in lines:
        s = line.strip()
        if not s:
            parsed.append(("blank", "", None))
        elif m := _TASK.match(s):
            parsed.append(("task", m.group(2), m.group(1).lower() == "x"))
        elif m := _BULLET.match(s):
            parsed.append(("bullet", m.group(1), None))
        elif m := _ORDERED.match(s):
            parsed.append(("ordered", m.group(1), None))
        elif m := _HEADING.match(s):
            parsed.append(("heading", m.group(2), len(m.group(1))))
        else:
            parsed.append(("text", s, None))

    if not any(k in ("task", "bullet", "ordered", "heading") for k, _, _ in parsed):
        return None
```

```
    def next_is_task(idx: int) -> bool:
        return idx < len(parsed) and parsed[idx][0] == "task"

    # "header": Notesnook's task list (title bar and checked/total count).
    # "simple": the plain checklist, no header, so a line above stays as text.
    header_style = get_todo_style() == "header"
    item_class = "checklist--item" if header_style else "simple-checklist--item"
    list_class = "checklist" if header_style else "simple-checklist"

    out = []
    pending_title = None  # line directly above a checklist -> its header title
    i = 0
    while i < len(parsed):
        kind, text, extra = parsed[i]
        if kind == "blank":
            i += 1
        elif kind == "heading":
            if header_style and next_is_task(i + 1) and _plain_title(text):
                pending_title = text
            else:
                out.append(f"<h{extra}>{text}</h{extra}>")
            i += 1
        elif kind in ("task", "bullet", "ordered"):
            items = []
            while i < len(parsed) and parsed[i][0] == kind:
                _, t, checked = parsed[i]
                if kind == "task":
                    # Same classes Notesnook itself writes. Without the item
                    # class, the task list header counts 0 items.
                    cls = f"{item_class} checked" if checked else item_class
                    items.append(f'<li class="{cls}">{t}</li>')
                else:
                    items.append(f"<li>{t}</li>")
                i += 1
            opener = {
                "task": f'<ul class="{list_class}"'
                + (_title_attr(pending_title) if header_style else "")
                + ">",
                "bullet": "<ul>",
                "ordered": "<ol>",
            }[kind]
            closer = "</ol>" if kind == "ordered" else "</ul>"
            out.append(opener + "".join(items) + closer)
            pending_title = None
        else:  # run of plain text lines -> one paragraph
            run = []
            while i < len(parsed) and parsed[i][0] == "text":
                run.append(parsed[i][1])
                i += 1
            if header_style and next_is_task(i) and _plain_title(run[-1]):
                pending_title = run.pop()
            if run:
                out.append("<p>" + "<br>".join(run) + "</p>")
    return "".join(out)


def telegram_html_to_note_html(tg_html: str) -> str:
    """Turn Telegram's HTML (tags already balanced and escaped) into note HTML.

    Markdown-style lists, todos and headings become real HTML lists when it is
    safe to (see _structured_html). Otherwise newlines become <br> outside
    <pre> blocks. We deliberately avoid splitting into multiple <p> tags in
    that case: a bold span can cross a blank line, and Notesnook turns the
    whole note into a code block if any tags are unbalanced.
    """
```

```
    tg_html = _TG_ONLY_TAGS.sub("", tg_html.strip())
    structured = _structured_html(tg_html)
    if structured is not None:
        return structured
    parts = _PRE_BLOCK.split(tg_html)
    out = [
        part if part.startswith("<pre>") else part.replace("\n", "<br>")
        for part in parts
    ]
    body = "".join(out)
    if any(tag in body for tag in _BLOCK_TAGS):
        return body  # block elements can't live inside <p>
    return f"<p>{body}</p>"


def generic_title(msg: Message) -> str:
    """e.g. 'Telegram note, Oct 15, 8:15 AM' in the BOT_TIMEZONE zone."""
    dt = msg.date
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    d = dt.astimezone(ZoneInfo(TIMEZONE))
    return f"Telegram note, {d:%b} {d.day}, {d.hour % 12 or 12}:{d.minute:02d} {d:%p}"


async def embed_attachment(msg: Message) -> tuple[str | None, str | None]:
    """Download a photo, image file or text file and return (html, None).

    Mirrors the Notesnook email worker: images become base64 data: URIs, text
    files become code blocks. Anything else, or anything over
    ATTACHMENT_SIZE_LIMIT, returns (None, reason) and gets a placeholder.
    """
    if not EMBED_MEDIA:
        return None, "embedding is turned off"

    name = None
    if msg.photo:
        tg_file, mime, kind = msg.photo[-1], "image/jpeg", "image"
    elif msg.document:
        tg_file = msg.document
        mime = (tg_file.mime_type or "").lower()
        name = tg_file.file_name or "file"
        ext = os.path.splitext(name)[1].lower()
        if mime in _IMAGE_MIMES:
            kind = "image"
        elif mime.startswith("text/") or mime in _TEXT_MIMES or ext in _TEXT_EXTS:
            kind = "text"
        else:
            return None, "only images and text files can be embedded"
    else:
        return None, "only photos, images and text files can be embedded"

    limit_mb = ATTACHMENT_SIZE_LIMIT // (1024 * 1024)
    if tg_file.file_size and tg_file.file_size > ATTACHMENT_SIZE_LIMIT:
        return None, f"too big ({tg_file.file_size / 1048576:.1f} MB, limit {limit_mb} MB)"
    try:
        downloaded = await tg_file.get_file()
        data = bytes(await downloaded.download_as_bytearray())
    except Exception as e:  # Telegram API or network error
        return None, f"download failed ({type(e).__name__})"
    if len(data) > ATTACHMENT_SIZE_LIMIT:
        return None, f"too big (limit {limit_mb} MB)"

    heading = f"<h3>{html.escape(name)}</h3>" if name else ""
    if kind == "image":
        b64 = base64.b64encode(data).decode("ascii")
```

```
        return f'{heading}<p><img src="data:{mime};base64,{b64}" /></p>', None
    if b"\x00" in data:
        return None, "file looks binary, not text"
    text = data.decode("utf-8", errors="replace")
    lang = _CODE_LANGUAGES.get(os.path.splitext(name or "")[1].lower())
    code_open = f'<code class="language-{lang}">' if lang else "<code>"
    return f"{heading}<pre>{code_open}{html.escape(text, quote=False)}</code></pre>", None


def build_note(
    msg: Message,
    attachment_html: str | None = None,
    problem: str | None = None,
    text_override: tuple[str, str] | None = None,
) -> tuple[str, str] | None:
    """Return (title, html) for a message, or None if there's nothing to save.

    attachment_html is an embedded photo/file block from embed_attachment; when
    a message has media but none is given, a placeholder line is added instead
    (with `problem` as the reason it wasn't embedded).
    """
    label = media_label(msg)
    plain = (msg.text or msg.caption or "").strip()
    rich = (msg.text_html if msg.text else msg.caption_html) or ""
    if text_override is not None:  # message with its #tag line removed
        plain, rich = text_override

    if not plain and not label:
        return None

    body_parts = []
    if plain:
        body_parts.append(telegram_html_to_note_html(rich))
    if label:
        if attachment_html:
            body_parts.append(attachment_html)
        else:
            why = f", not embedded: {html.escape(problem)}" if problem else ""
            body_parts.append(f"<p><i>[{label} attached in Telegram{why}]</i></p>")

    if plain:
        first_line = plain.splitlines()[0].strip()
        if _LIST_START.match(first_line):
            # Starts with a list item: a generic title avoids duplicating it.
            title = generic_title(msg)
        else:
            first_line = _HEADING_MARK.sub("", first_line) or first_line
            first_line = first_line.rstrip(":").strip() or first_line
            title = (first_line[:80] + "…") if len(first_line) > 80 else first_line
    else:
        file_name = msg.document.file_name if msg.document else None
        title = file_name or f"{label} from Telegram"

    return title, "".join(body_parts)


_gpg_home: str | None = None
_gpg_fpr: str | None = None
_gpg_lock = threading.Lock()


def _gpg(*args: str, stdin: bytes | None = None) -> bytes:
    proc = subprocess.run(
        ["gpg", "--batch", "--yes", "--homedir", _gpg_home, *args],
        input=stdin,
```

```
        capture_output=True,
        timeout=30,
    )
    if proc.returncode != 0:
        raise RuntimeError(f"gpg failed: {proc.stderr.decode(errors='replace')[:200]}")
    return proc.stdout


def _ensure_public_key(refresh: bool = False) -> str:
    """Fetch the account's inbox public key from the server and import it."""
    global _gpg_home, _gpg_fpr
    with _gpg_lock:
        if _gpg_fpr and not refresh:
            return _gpg_fpr
        if _gpg_home is None:
            _gpg_home = tempfile.mkdtemp(prefix="gnupg-")
            os.chmod(_gpg_home, 0o700)
        r = requests.get(
            f"{SERVER_URL}/inbox/public-encryption-key",
            headers={"Authorization": INBOX_KEY},
            timeout=20,
        )
        r.raise_for_status()
        key_text = r.json()["key"] if r.headers.get(
            "content-type", ""
        ).startswith("application/json") else r.text
        _gpg("--import", stdin=key_text.encode())
        listing = _gpg("--list-keys", "--with-colons").decode()
        _gpg_fpr = next(
            line.split(":")[9] for line in listing.splitlines() if line.startswith("fpr:")
        )
        return _gpg_fpr


def _encrypt(plaintext: bytes) -> str:
    fpr = _ensure_public_key()
    return _gpg(
        "--trust-model", "always", "--armor", "--cipher-algo", "AES256",
        "-r", fpr, "-e",
        stdin=plaintext,
    ).decode()


TAG_NAME = re.compile(r"^[\w-]{1,40}$")
TAG_ID = re.compile(r"^[A-Za-z0-9]{12,64}$")
TAG_TOKEN = re.compile(r"^#[\w-]{1,40}$")
TAGS_FILE = os.environ.get("TAGS_FILE") or (
    "/data/tags.json" if os.path.isdir("/data") else "tags.json"
)
# True when the tag file sits on a mounted Fly volume, so it survives redeploys.
TAGS_PERSISTENT = os.path.ismount(os.path.dirname(os.path.abspath(TAGS_FILE)))
HISTORY_LIMIT = 500

_tags_lock = threading.Lock()
_tags_state: dict | None = None


def _now_iso() -> str:
    return datetime.now(ZoneInfo(TIMEZONE)).isoformat(timespec="seconds")


def _save_tags_locked() -> None:
    """Write the tag file atomically: temp file, then rename over the old one."""
    folder = os.path.dirname(os.path.abspath(TAGS_FILE))
    os.makedirs(folder, exist_ok=True)
```

```
    tmp = TAGS_FILE + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(_tags_state, f, indent=2, ensure_ascii=False)
    os.replace(tmp, TAGS_FILE)


def _tags_locked() -> dict:
    """Load the tag file on first use. Caller must hold _tags_lock."""
    global _tags_state
    if _tags_state is not None:
        return _tags_state
    state: dict = {}
    existed = os.path.exists(TAGS_FILE)
    if existed:
        try:
            with open(TAGS_FILE, encoding="utf-8") as f:
                state = json.load(f)
            if not isinstance(state, dict) or not isinstance(state.get("tags", {}), dict):
                raise ValueError("bad tag file")
        except (OSError, ValueError):
            try:  # keep the unreadable file instead of overwriting it
                os.replace(TAGS_FILE, TAGS_FILE + ".corrupt")
            except OSError:
                pass
            state, existed = {}, False
    state.setdefault("tags", {})
    state.setdefault("history", [])
    _tags_state = state
    if not existed:  # first run: optionally seed from the NOTESNOOK_TAGS secret
        try:
            seed = json.loads(os.environ.get("NOTESNOOK_TAGS", "") or "{}")
        except ValueError:
            seed = {}
        for name, tag_id in (seed.items() if isinstance(seed, dict) else []):
            name = str(name).lower().lstrip("#")
            if TAG_NAME.match(name) and TAG_ID.match(str(tag_id)):
                state["tags"][name] = str(tag_id)
                state["history"].append({"at": _now_iso(), "action": "seed", "name": name})
        try:
            _save_tags_locked()
        except OSError:
            pass
    return state


def get_tags() -> dict:
    with _tags_lock:
        return dict(_tags_locked()["tags"])


def tags_snapshot() -> dict:
    with _tags_lock:
        state = _tags_locked()
        return {"tags": dict(state["tags"]), "history": list(state["history"])}


def _log(state: dict, action: str, name: str) -> None:
    state["history"].append({"at": _now_iso(), "action": action, "name": name})
    del state["history"][:-HISTORY_LIMIT]


def set_tag(name: str, tag_id: str) -> str:
    with _tags_lock:
        state = _tags_locked()
        action = "updated" if name in state["tags"] else "added"
```

```
        state["tags"][name] = tag_id
        _log(state, action, name)
        _save_tags_locked()
        return action


def remove_tag(name: str) -> bool:
    with _tags_lock:
        state = _tags_locked()
        if name not in state["tags"]:
            return False
        del state["tags"][name]
        _log(state, "removed", name)
        _save_tags_locked()
        return True


TODO_STYLES = ("header", "simple")


def get_todo_style() -> str:
    """How todos are saved: "header" (task list with title and count, the default)
    or "simple" (plain checklist). Stored in the same file as the tags."""
    with _tags_lock:
        style = _tags_locked().get("todo_style")
    return style if style in TODO_STYLES else "header"


def set_todo_style(style: str) -> None:
    with _tags_lock:
        state = _tags_locked()
        state["todo_style"] = style
        _log(state, "todo style", style)
        _save_tags_locked()


def replace_tags(tags: dict) -> int:
    with _tags_lock:
        state = _tags_locked()
        state["tags"] = dict(tags)
        _log(state, "restored", f"{len(tags)} tags")
        _save_tags_locked()
        return len(tags)


def extract_tag_line(msg: Message) -> tuple[list[str], tuple[str, str] | None]:
    """Find a #tag line at the end of a message.

    Returns (tag names, text_override). text_override is (plain, rich) with the
    tag line removed, or None when the line should stay in the note (it is
    only removed when every tag on it is known and the line can be cut cleanly).
    Needs at least one other line of text, or media, so a message that is just
    "#groceries" is saved as a normal note.
    """
    plain = msg.text or msg.caption or ""
    lines = plain.rstrip().split("\n")
    if len(lines) < 2 and not media_label(msg):
        return [], None
    tokens = lines[-1].split()
    if not tokens or not all(TAG_TOKEN.match(t) for t in tokens):
        return [], None
    rest_plain = "\n".join(lines[:-1]).strip()
    if not rest_plain and not media_label(msg):
        return [], None
    names = list(dict.fromkeys(t[1:].lower() for t in tokens))
```

```
    if not all(n in get_tags() for n in names):
        return names, None
    rich = ((msg.text_html if msg.text else msg.caption_html) or "").rstrip()
    rich_lines = rich.split("\n")
    last_rich = html.unescape(re.sub(r"<[^>]+>", "", rich_lines[-1])).strip()
    if len(rich_lines) != len(lines) or last_rich != lines[-1].strip():
        return names, None
    return names, (rest_plain, "\n".join(rich_lines[:-1]).strip())


def send_to_notesnook(
    title: str, content_html: str, tag_ids: list[str] | None = None
) -> requests.Response:
    payload = {
        "title": title,
        "type": "note",
        "source": "telegram",
        "version": 1,
        "content": {"type": "html", "data": content_html},
    }
    if tag_ids:
        payload["tagIds"] = tag_ids
    if not SERVER_URL:  # hosted Notesnook inbox: the proxy encrypts for us
        return requests.post(
            HOSTED_INBOX_URL,
            json=payload,
            headers={"Authorization": INBOX_KEY},
            timeout=20,
        )

    plaintext = json.dumps(payload).encode("utf-8")
    try:
        cipher = _encrypt(plaintext)
    except Exception:
        _ensure_public_key(refresh=True)  # key may have been rotated; retry once
        cipher = _encrypt(plaintext)
    envelope = {"v": 1, "alg": "pgp-aes256", "cipher": cipher}
    return requests.post(
        f"{SERVER_URL}/inbox/items",
        json=envelope,
        headers={"Authorization": INBOX_KEY},
        timeout=20,
    )


async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.effective_message
    if msg is None or update.effective_user.id != ALLOWED_USER_ID:
        return  # ignore everyone else

    if await handle_pending(update, context):
        return  # this message answered a /tag or /untag prompt

    attachment_html, problem = None, None
    if media_label(msg):
        attachment_html, problem = await embed_attachment(msg)

    tag_names, text_override = extract_tag_line(msg)
    known = get_tags()
    applied = [n for n in tag_names if n in known]
    unknown = [n for n in tag_names if n not in known]

    note = build_note(msg, attachment_html, problem, text_override)
    if note is None:
```

```
        await msg.reply_text("Nothing to save in that message.")
        return
    title, content_html = note

    if attachment_html and len(content_html.encode("utf-8")) > NOTE_SIZE_LIMIT:
        attachment_html, problem = None, "the note would be too large"
        title, content_html = build_note(msg, None, problem, text_override)

    tag_ids = [known[n] for n in applied]
    try:
        resp = await asyncio.to_thread(send_to_notesnook, title, content_html, tag_ids)
    except requests.RequestException as e:
        await msg.reply_text(f"Network error: {e}")
        return

    if resp.status_code == 200:
        suffix = ""
        if media_label(msg):
            suffix = " (media embedded)" if attachment_html else f" (media not embedded: {problem})"
        if applied:
            suffix += f" (tags: {', '.join(applied)})"
        text = f"✓ Saved to Notesnook{suffix}"
        if unknown:
            names = ", ".join("#" + n for n in unknown)
            text += f"\nTag not found: {names}. Add it with /tag name id"
        await msg.reply_text(text)
    else:
        await msg.reply_text(f"Failed ({resp.status_code}): {resp.text[:300]}")


HELP_TEXT = """Telegram -> Notesnook bridge

Send this bot a message to save it as a note in Notesnook. Notes appear in Notesnook after its next sync.

COMMANDS
/help   show this guide
/setup  how to connect the bot to Notesnook (default or self-hosted)
/ver    show the bot version and where notes are going
/html   save raw HTML as a note (see HTML below)
/callout  save a callout box (see CALLOUTS below)
/todostyle  switch todos between a header task list and a simple checklist
/tag    save a tag: /tag name id (or just /tag, and the bot asks)
/untag  forget a tag: /untag name (or just /untag)
/cancel stop a /tag, /untag or /callout prompt
/tags   list saved tags
/tagsfile  send the saved tags as a backup file

LISTS, TODOS AND HEADINGS
Start a line with one of these:
- item (or * item, • item)  ->  bullet list
1. item (or 1) item)  ->  numbered list
- [ ] item  ->  todo
- [x] item  ->  completed todo
# Heading (up to ###### Heading)  ->  heading

Example:
Groceries:
- [ ] milk
- [x] eggs

A line directly above a todo list, with no blank line between, becomes the title of that checklist. In the
example, "Groceries" is the checklist title.

TODO STYLE
Todos are saved in one of two styles. Send /todostyle to switch between them (or /todostyle header,
```

```
/todostyle simple). The choice is remembered.
• header (default): Notesnook's task list, with a title bar, a progress bar and a checked/total count.
• simple: a plain checklist with no header or count. The line above the todos stays in the note as normal
text.

NOTE TITLES
The first line of a message is the note title. A message that starts with a list item gets a title with
the date and time instead, like "Telegram note, Oct 15, 8:15 AM".

FORMATTING
Select text and use Telegram's formatting options to apply bold, italic, underline, strikethrough, links,
monospace (code) and quotes. All of these carry over to the note. Spoiler text is saved as plain text.

HTML
In a normal message, tags typed by hand, such as <b>, are not interpreted and appear in the note as plain
text. To save real HTML, start the message with /html:
/html <h1>Groceries</h1><ul class="checklist"><li class="checklist--item">milk</li><li class="checklist--
item checked">eggs</li></ul>
The HTML is sent exactly as typed. The note title is the first heading (<h1> to <h6>), or the date and
time if there is none. Every tag must be closed, or the bot refuses to save it (Notesnook would turn the
note into a code block). A #tag on the last line works as usual. Checklist items need class="checklist--
item", plus "checked" when done.

CALLOUTS
Send /callout on its own and the bot asks for the type, the title and the text, one message each (send
skip to leave one out, /cancel to stop). Or do it in one message: start with /callout, then an optional
type and a title, then the body on the next lines:
/callout warning Back up first
Export your notes before upgrading.
Types: note, abstract, summary, tldr, info, todo, tip, hint, important, success, check, done, question,
help, faq, warning, warn, caution, attention, failure, fail, missing, danger, error, bug, example, quote,
cite. With no type, or an unknown one, the callout is info. With no title, the type name is used. The body
can use lists, todos and Telegram formatting. The whole message becomes the callout, and the note title is
the callout title. A #tag on the last line works as usual.

TAGS
Put tags on the last line of a message, after your text:
Groceries:
- [ ] milk

#groceries #home

The tag line is removed from the note and the matching Notesnook tags are applied. Tags only count on the
last line, and the message needs other text above it.

Notesnook can't list your tags, so save each one once. In Notesnook, right-click a tag and choose Copy ID,
then send:
/tag groceries 67aecf3b9e1398484554bc90

A tag the bot doesn't know stays in the note as text, and the reply says it was not found. After every
/tag and /untag the bot sends you the tag file. To restore it, send that file back with the caption
/restore.

PHOTOS AND FILES
• Photos and image files (JPEG, PNG, GIF, WebP) are embedded in the note.
• Text files (.txt, .md, .csv, .json, code and similar) are added as code blocks. Common types (.py, .js,
.ts, .json, .yaml, .sh, .sql, .md, .css, .toml, .ini, .csv, .html, .xml) get syntax highlighting.
• Files can be up to 5 MB each.
• Videos, voice notes and other file types are not embedded. The note shows a placeholder line, plus the
caption if there is one.
• Embedded images are part of the note itself, not Notesnook attachments. To save a real attachment, share
the file to Notesnook from your phone's share sheet.
• Each photo in an album becomes its own note.

LIMITS
```

```
• Lists can't be nested, and numbering always starts at 1.
• A message that contains a code block, a quote, or bold text spanning several lines is saved as simple
text, without list conversion."""


SETUP_TEXT = """Setup: connecting the bot to Notesnook (v1.4.0)

The bot can send notes to ONE of two places. Which one is decided by a single setting,
NOTESNOOK_SERVER_URL.

OPTION A: DEFAULT NOTESNOOK (hosted)
Use this if you sync with Notesnook's own servers.
1. In Notesnook: Settings > Account > Inbox, turn it on and create an Inbox API key.
2. Set the key on the bot:
   fly secrets set NOTESNOOK_INBOX_KEY=<your key>
3. Leave NOTESNOOK_SERVER_URL unset. If it was set before, remove it:
   fly secrets unset NOTESNOOK_SERVER_URL
4. /ver should say: Destination: hosted Notesnook inbox
Notesnook's server encrypts your notes for you.

OPTION B: SELF-HOSTED NOTESNOOK
Use this if you run your own sync server.
1. Log in to the Notesnook app that is connected to YOUR server (not the default one). The key must come
from this account.
2. Open Settings > Account > Inbox, turn it on, and create an Inbox API key. Copy it so you can paste it
in the next step.
3. Set both settings on the bot. Use your own server address, with no slash at the end:
   fly secrets set NOTESNOOK_SERVER_URL=https://notesnook.example.com
   fly secrets set NOTESNOOK_INBOX_KEY=<your self-hosted key>
4. Deploy the new code: fly deploy
5. /ver should show your server address under Destination.
6. Send a test message, then sync Notesnook.

IMPORTANT FOR SELF-HOSTED
- A key from the default Notesnook account will NOT work on your server (the bot reports 401). Create a
new key on the self-hosted account.
- The bot encrypts every note with your account's public key before sending. It fetches that key from your
server on its own using the Inbox key. You do not paste any other key.
- The server address is the main sync address (the notesnook. host), not the auth, events, monograph or
attachments addresses.
- If you reset your self-hosted account's encryption keys, create a new Inbox key and set it again. The
bot re-fetches the public key automatically.

TROUBLESHOOTING
401  wrong or missing key, or key from the other account
404  wrong server address
Network error  server is unreachable
Saved but no note  sync Notesnook; check you are logged in to the matching account"""


def _authorized(update: Update) -> bool:
    user = update.effective_user
    return user is not None and user.id == ALLOWED_USER_ID


def _chunks(text: str, limit: int = 3800) -> list[str]:
    """Split text at blank lines into pieces under Telegram's 4096-character cap."""
    parts, current = [], ""
    for block in text.split("\n\n"):
        if current and len(current) + len(block) + 2 > limit:
            parts.append(current)
            current = block
        else:
            current = f"{current}\n\n{block}" if current else block
    if current:
```

```
        parts.append(current)
    return parts


async def cmd_help(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
        for part in _chunks(HELP_TEXT):
            await update.effective_message.reply_text(part)


async def cmd_setup(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
        for part in _chunks(SETUP_TEXT):
            await update.effective_message.reply_text(part)


async def send_backup(msg: Message, reason: str) -> None:
    """Send the tag file to the chat, so Telegram holds an off-Fly copy."""
    try:
        data = json.dumps(tags_snapshot(), indent=2, ensure_ascii=False).encode("utf-8")
        stamp = datetime.now(ZoneInfo(TIMEZONE)).strftime("%Y-%m-%d_%H%M")
        await msg.reply_document(
            document=io.BytesIO(data),
            filename=f"notesnook-tags-{stamp}.json",
            caption=f"Tag backup ({reason}). To restore, send this file back with the caption /restore",
        )
    except Exception as e:  # never let a failed backup break the command
        await msg.reply_text(f"Could not send the tag backup ({type(e).__name__}).")


PENDING = "pending"  # key in context.user_data: a command waiting for your next message
PENDING_SECONDS = 300  # a forgotten prompt stops swallowing messages after 5 minutes
ASK_NAME = "Which tag name? Use letters, numbers, - and _. Send /cancel to stop."
ASK_ID = (
    'Now send the tag ID for "{name}". In Notesnook, right-click the tag and '
    "choose Copy ID. Send /cancel to stop."
)


def _wait_for(context: ContextTypes.DEFAULT_TYPE, cmd: str, **extra) -> None:
    context.user_data[PENDING] = {"cmd": cmd, "at": time.time(), **extra}


async def save_tag(msg: Message, name_raw: str, id_raw: str) -> bool:
    """Validate and save a tag, then send the backup. False if input was invalid."""
    name, tag_id = name_raw.lstrip("#").lower(), id_raw.strip()
    if not TAG_NAME.match(name):
        await msg.reply_text("Tag names can use letters, numbers, - and _ (up to 40).")
        return False
    if not TAG_ID.match(tag_id):
        await msg.reply_text("That doesn't look like a tag ID. Use Copy ID in Notesnook.")
        return False
    result = set_tag(name, tag_id)
    await msg.reply_text(f'Tag "{name}" {result}.')
    await send_backup(msg, f"after /tag {name}")
    return True


async def forget_tag(msg: Message, name_raw: str) -> bool:
    name = name_raw.lstrip("#").lower()
    if remove_tag(name):
        await msg.reply_text(f'Tag "{name}" forgotten. It is not deleted in Notesnook.')
        await send_backup(msg, f"after /untag {name}")
        return True
    await msg.reply_text(f'No saved tag named "{name}". Send /tags to see them.')
```

```
    return False


async def handle_pending(update: Update, context: ContextTypes.DEFAULT_TYPE) -> bool:
    """Use the message as the answer to a /tag, /untag or /callout prompt. True if it was."""
    pend = context.user_data.get(PENDING)
    if not pend:
        return False
    msg = update.effective_message
    text = (msg.text or "").strip()
    if time.time() - pend["at"] > PENDING_SECONDS or not text:
        context.user_data.pop(PENDING, None)  # expired, or not text: save it as a note
        return False
    if pend["cmd"] == "tag_name":
        name = text.lstrip("#").lower()
        if not TAG_NAME.match(name):
            await msg.reply_text("Tag names can use letters, numbers, - and _ (up to 40). Try again, or
/cancel.")
        else:
            _wait_for(context, "tag_id", name=name)
            await msg.reply_text(ASK_ID.format(name=name))
    elif pend["cmd"] == "tag_id":
        if await save_tag(msg, pend["name"], text):
            context.user_data.pop(PENDING, None)
        else:
            await msg.reply_text("Try again, or /cancel.")
    elif pend["cmd"] == "untag":
        context.user_data.pop(PENDING, None)
        await forget_tag(msg, text)
    elif pend["cmd"] == "callout_type":
        word = text.lower()
        if word in ("skip", "-"):
            word = "info"
        if word not in _CALLOUT_TYPES:
            await msg.reply_text(
                "That is not a callout type. Try note, info, tip, warning or danger "
                "(full list in /help), send skip for info, or /cancel."
            )
        else:
            _wait_for(context, "callout_title", ctype=word)
            await msg.reply_text(ASK_CALLOUT_TITLE)
    elif pend["cmd"] == "callout_title":
        title = "" if text.lower() == "skip" else " ".join((msg.text_html or "").split())
        _wait_for(context, "callout_body", ctype=pend["ctype"], title=title)
        await msg.reply_text(ASK_CALLOUT_BODY)
    elif pend["cmd"] == "callout_body":
        context.user_data.pop(PENDING, None)
        if text.lower() == "skip":
            tag_names, body = [], ""
        else:
            tag_names, body = _callout_tags(msg.text_html or "")
        note_title, content_html = _callout_html(pend["ctype"], pend["title"], body)
        await _save_callout(msg, note_title, content_html, tag_names)
    return True


async def cmd_cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    had = context.user_data.pop(PENDING, None)
    await update.effective_message.reply_text("Cancelled." if had else "Nothing to cancel.")


async def cmd_tag(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """/tag name id saves a tag. With less, the bot asks for the rest."""
```

```
    if not _authorized(update):
        return
    msg = update.effective_message
    args = context.args or []
    context.user_data.pop(PENDING, None)
    if len(args) == 2:
        await save_tag(msg, args[0], args[1])
    elif len(args) == 1:
        name = args[0].lstrip("#").lower()
        if not TAG_NAME.match(name):
            await msg.reply_text("Tag names can use letters, numbers, - and _ (up to 40).")
            return
        _wait_for(context, "tag_id", name=name)
        await msg.reply_text(ASK_ID.format(name=name))
    elif not args:
        _wait_for(context, "tag_name")
        await msg.reply_text(ASK_NAME)
    else:
        await msg.reply_text("Usage: /tag name id")


async def cmd_untag(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """/untag name forgets a tag. With no name, the bot lists them and asks."""
    if not _authorized(update):
        return
    msg = update.effective_message
    args = context.args or []
    context.user_data.pop(PENDING, None)
    if len(args) == 1:
        await forget_tag(msg, args[0])
    elif not args:
        tags = get_tags()
        if not tags:
            await msg.reply_text("No tags saved yet.")
            return
        _wait_for(context, "untag")
        await msg.reply_text(
            "Which tag should be forgotten? Send its name, or /cancel.\n\n"
            + "\n".join(sorted(tags))
        )
    else:
        await msg.reply_text("Usage: /untag name")


async def cmd_tags(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    tags = get_tags()
    if tags:
        lines = [f"{n}  {i[:8]}…{i[-4:]}" for n, i in sorted(tags.items())]
        text = "Saved tags:\n" + "\n".join(lines)
    else:
        text = "No tags saved yet. Add one with /tag name id."
    if not TAGS_PERSISTENT:
        text += (
            "\n\nWarning: no Fly volume is mounted, so tags are lost when the bot "
            "redeploys. Restore from your last backup file with /restore."
        )
    await update.effective_message.reply_text(text)


async def cmd_tagsfile(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
        await send_backup(update.effective_message, "on request")
```

```
async def cmd_restore(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """A tag backup file sent with the caption /restore replaces the saved tags."""
    if not _authorized(update):
        return
    msg = update.effective_message
    doc = msg.document
    if doc is None or (doc.file_size or 0) > 1_000_000:
        await msg.reply_text("Send a tag backup file (under 1 MB) with the caption /restore.")
        return
    try:
        tg_file = await doc.get_file()
        data = json.loads(bytes(await tg_file.download_as_bytearray()).decode("utf-8"))
        raw = data["tags"]
        if not isinstance(raw, dict):
            raise ValueError("tags must be an object")
        clean = {
            str(n).lower(): str(i)
            for n, i in raw.items()
            if TAG_NAME.match(str(n).lower()) and TAG_ID.match(str(i))
        }
        if raw and not clean:
            raise ValueError("no valid tags")
    except Exception:
        await msg.reply_text("Could not read that file. Send a tag backup file with the caption
/restore.")
        return
    count = replace_tags(clean)
    await msg.reply_text(f"Restored {count} tags.")


async def cmd_ver(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    lines = [f"Telegram -> Notesnook bridge v{__version__}"]
    if region := os.environ.get("FLY_REGION"):
        lines.append(f"Running on Fly.io (region: {region})")
    lines.append(f"Destination: {SERVER_URL or 'hosted Notesnook inbox'}")
    lines.append(
        f"Saved tags: {len(get_tags())} "
        f"({'on a Fly volume' if TAGS_PERSISTENT else 'NOT persistent: no volume mounted'})"
    )
    lines.append(f"Time zone for generic titles: {TIMEZONE}")
    lines.append(f"Media embedding: {'on' if EMBED_MEDIA else 'off'}")
    lines.append(f"Todo style: {get_todo_style()} (change with /todostyle)")
    await update.effective_message.reply_text("\n".join(lines))


_VOID_TAGS = {
    "area", "base", "br", "col", "embed", "hr", "img", "input", "link", "meta",
    "param", "source", "track", "wbr",
}
_HTML_TAG = re.compile(r"<(/?)([a-zA-Z][a-zA-Z0-9-]*)\b[^>]*?(/?)>")
_HTML_HEADING = re.compile(r"<h[1-6]\b[^>]*>(.*?)</h[1-6]>", re.IGNORECASE | re.DOTALL)
_HTML_COMMAND = re.compile(r"^/html(?:@\w+)?[ \t]*\n?", re.IGNORECASE)


def _html_balanced(raw: str) -> bool:
    """True if every non-void tag in raw HTML is closed in the right order.

    Notesnook saves a note as a code block when tags are unbalanced, so the bot
    checks first and refuses instead of saving a mangled note.
    """
    stripped = re.sub(r"<!--.*?-->", "", raw, flags=re.DOTALL)
    stack = []
```

```
    for m in _HTML_TAG.finditer(stripped):
        closing, name, self_closing = m.group(1), m.group(2).lower(), m.group(3)
        if name in _VOID_TAGS or self_closing:
            continue
        if closing:
            if not stack or stack.pop() != name:
                return False
        else:
            stack.append(name)
    return not stack


def _html_title(msg: Message, raw: str) -> str:
    """Title for a /html note: the first heading's text, else a generic title."""
    if m := _HTML_HEADING.search(raw):
        plain = html.unescape(re.sub(r"<[^>]+>", "", m.group(1))).strip().rstrip(":").strip()
        if plain:
            return (plain[:80] + "…") if len(plain) > 80 else plain
    return generic_title(msg)


async def cmd_html(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """/html <raw HTML>: save the HTML exactly as typed (no escaping)."""
    msg = update.effective_message
    if msg is None or not _authorized(update):
        return
    raw = _HTML_COMMAND.sub("", msg.text or "", count=1).strip()
    if not raw:
        await msg.reply_text(
            "Send /html followed by your HTML, for example:\n"
            "/html <h1>Groceries</h1><ul class=\"checklist\">"
            "<li class=\"checklist--item\">milk</li></ul>"
        )
        return

    # Optional #tag line at the end, same rule as normal notes.
    lines = raw.split("\n")
    tag_names: list[str] = []
    if len(lines) >= 2:
        tokens = lines[-1].split()
        if tokens and all(TAG_TOKEN.match(t) for t in tokens):
            tag_names = list(dict.fromkeys(t[1:].lower() for t in tokens))
    known = get_tags()
    applied = [n for n in tag_names if n in known]
    unknown = [n for n in tag_names if n not in known]
    if tag_names and not unknown:
        raw = "\n".join(lines[:-1]).strip()  # remove the tag line

    if not _html_balanced(raw):
        await msg.reply_text(
            "Not saved: the HTML has an unclosed or mismatched tag. "
            "Notesnook would turn the whole note into a code block. "
            "Fix the tags and send it again."
        )
        return

    title = _html_title(msg, raw)
    try:
        resp = await asyncio.to_thread(
            send_to_notesnook, title, raw, [known[n] for n in applied]
        )
    except requests.RequestException as e:
        await msg.reply_text(f"Network error: {e}")
        return
```

```
    if resp.status_code == 200:
        text = "✓ Saved to Notesnook (raw HTML)"
        if applied:
            text += f" (tags: {', '.join(applied)})"
        if unknown:
            names = ", ".join("#" + n for n in unknown)
            text += f"\nTag not found: {names}. Add it with /tag name id"
        await msg.reply_text(text)
    else:
        await msg.reply_text(f"Failed ({resp.status_code}): {resp.text[:300]}")


# Callout types Notesnook knows (from its editor source). Unknown types fall back to info.
_CALLOUT_TYPES = {
    "note", "abstract", "summary", "tldr", "info", "todo", "tip", "hint",
    "important", "success", "check", "done", "question", "help", "faq",
    "warning", "warn", "caution", "attention", "failure", "fail", "missing",
    "danger", "error", "bug", "example", "quote", "cite",
}
_CALLOUT_COMMAND = re.compile(r"^/callout(?:@\w+)?[ \t]*\n?", re.IGNORECASE)
_CALLOUT_FIRST = re.compile(r"^([A-Za-z]+)(?:[ \t]+(.*))?$")


def build_callout(rich: str) -> tuple[str, str] | None:
    """Turn the text after /callout into (note title, callout HTML).

    First line: optional type, then the title. Remaining lines: the body, which
    can use lists, todos and Telegram formatting like a normal message.
    Returns None when there is nothing after the command.
    """
    rest = _CALLOUT_COMMAND.sub("", rich.strip(), count=1).strip()
    if not rest:
        return None
    first, _, body = rest.partition("\n")
    first, body = first.strip(), body.strip()
    if not (_balanced(first) and _balanced(body)):
        # Formatting runs across the title/body split; keep the text, drop the
        # tags so the HTML stays balanced (Notesnook rejects unbalanced HTML).
        first, body = _TAG.sub("", first), _TAG.sub("", body)

    ctype, title = "info", first
    if (m := _CALLOUT_FIRST.match(first)) and m.group(1).lower() in _CALLOUT_TYPES:
        ctype, title = m.group(1).lower(), (m.group(2) or "").strip()
    return _callout_html(ctype, title, body)


def _callout_html(ctype: str, title: str, body: str) -> tuple[str, str]:
    """(note title, callout HTML) from a type, a title and a body (Telegram HTML)."""
    if not title:
        title = ctype.upper()  # Notesnook's default title is the type name
    body_html = telegram_html_to_note_html(_TG_ONLY_TAGS.sub("", body)) if body else "<p></p>"
    callout = (
        f'<div class="callout" data-callout-type="{ctype}">'
        f"<h4>{title}</h4>{body_html}</div>"
    )
    plain_title = _plain_title(title) or ctype.upper()
    note_title = (plain_title[:80] + "…") if len(plain_title) > 80 else plain_title
    return note_title, callout


def _callout_tags(rich: str) -> tuple[list[str], str]:
    """Find tags at the end of a callout message (Telegram HTML in, HTML out).

    Accepts a last line made only of #tags (removed when they are all saved
    tags), or saved #tags at the end of the last line, so
```

```
    "/callout tip Title #home" works too. Returns (tag names, text without them).
    """
    lines = rich.rstrip().split("\n")
    known = get_tags()
    last = lines[-1]
    tokens = html.unescape(_TAG.sub("", last)).split()
    if tokens and "<" not in last and all(TAG_TOKEN.match(t) for t in tokens):
        names = list(dict.fromkeys(t[1:].lower() for t in tokens))
        if all(n in known for n in names):
            return names, "\n".join(lines[:-1]).strip()
        return names, rich  # an unknown tag: keep the text, the reply says so
    found: list[str] = []
    text = last
    while (m := re.search(r"(?:^|\s)#([\w-]{1,40})\s*$", text)) and m.group(1).lower() in known:
        found.insert(0, m.group(1).lower())
        text = text[: m.start()].rstrip()
    if found:
        return found, "\n".join(lines[:-1] + [text]).strip()
    return [], rich


ASK_CALLOUT_TYPE = (
    "Which callout type? For example: note, info, tip, warning, danger, success, "
    "question, example (the full list is in /help). Send skip for info, or /cancel to stop."
)
ASK_CALLOUT_TITLE = "What is the callout's title? Send skip to use the type name, or /cancel to stop."
ASK_CALLOUT_BODY = (
    "What is the callout's text? Lists, todos and Telegram formatting work. "
    "A #tag on the last line is applied. Send skip for no text, or /cancel to stop."
)


async def _save_callout(
    msg: Message, title: str, content_html: str, tag_names: list[str]
) -> None:
    """Send a callout note to Notesnook and report the result."""
    known = get_tags()
    applied = [n for n in tag_names if n in known]
    unknown = [n for n in tag_names if n not in known]
    try:
        resp = await asyncio.to_thread(
            send_to_notesnook, title, content_html, [known[n] for n in applied]
        )
    except requests.RequestException as e:
        await msg.reply_text(f"Network error: {e}")
        return

    if resp.status_code == 200:
        text = "✓ Saved to Notesnook (callout)"
        if applied:
            text += f" (tags: {', '.join(applied)})"
        if unknown:
            names = ", ".join("#" + n for n in unknown)
            text += f"\nTag not found: {names}. Add it with /tag name id"
        await msg.reply_text(text)
    else:
        await msg.reply_text(f"Failed ({resp.status_code}): {resp.text[:300]}")


async def cmd_callout(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """/callout [type] [title] then body lines. With nothing after it, the bot asks."""
    msg = update.effective_message
    if msg is None or not _authorized(update):
        return
    context.user_data.pop(PENDING, None)
```

```
    tag_names, rich = _callout_tags(msg.text_html or "")

    built = build_callout(rich)
    if built is None:
        _wait_for(context, "callout_type")
        await msg.reply_text(ASK_CALLOUT_TYPE)
        return
    title, content_html = built
    await _save_callout(msg, title, content_html, tag_names)


async def cmd_todostyle(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """/todostyle switches between the header task list and the simple checklist."""
    if not _authorized(update):
        return
    msg = update.effective_message
    args = [a.lower() for a in (context.args or [])]
    current = get_todo_style()
    if args and args[0] in TODO_STYLES:
        new = args[0]
    elif not args:
        new = "simple" if current == "header" else "header"  # no argument: toggle
    else:
        await msg.reply_text("Usage: /todostyle, /todostyle header or /todostyle simple")
        return
    if new != current:
        try:
            set_todo_style(new)
        except OSError as e:
            await msg.reply_text(f"Could not save the setting ({type(e).__name__}).")
            return
    desc = (
        "header: a task list with a title and a checked/total count"
        if new == "header"
        else "simple: a plain checklist, with no header or count"
    )
    note = "" if new != current else " (no change)"
    if not TAGS_PERSISTENT:
        note += "\nNo Fly volume is mounted, so this resets when the bot redeploys."
    await msg.reply_text(f"Todo style is now {desc}.{note}\nSend /todostyle again to switch.")


async def cmd_unknown(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
        await update.effective_message.reply_text("Unknown command. Send /help.")


async def post_init(app: Application):
    await app.bot.set_my_commands(
        [
            BotCommand("help", "How to use this bot"),
            BotCommand("setup", "Connect to default or self-hosted Notesnook"),
            BotCommand("html", "Save raw HTML as a note"),
            BotCommand("callout", "Save a callout (type, title, body)"),
            BotCommand("todostyle", "Switch todos: header or simple checklist"),
            BotCommand("tag", "Save a tag (the bot asks for details)"),
            BotCommand("untag", "Forget a saved tag"),
            BotCommand("tags", "List saved tags"),
            BotCommand("tagsfile", "Send the tag backup file"),
            BotCommand("ver", "Show the bot version"),
        ]
    )
```

```
def main():
    app = (
        Application.builder()
        .token(BOT_TOKEN)
        .read_timeout(60)  # file downloads can be slow
        .write_timeout(60)
        .post_init(post_init)
        .build()
    )
    app.add_handler(CommandHandler(["start", "help"], cmd_help))
    app.add_handler(CommandHandler("setup", cmd_setup))
    app.add_handler(CommandHandler("cancel", cmd_cancel))
    app.add_handler(CommandHandler("tag", cmd_tag))
    app.add_handler(CommandHandler("untag", cmd_untag))
    app.add_handler(CommandHandler("tags", cmd_tags))
    app.add_handler(CommandHandler("tagsfile", cmd_tagsfile))
    app.add_handler(CommandHandler("ver", cmd_ver))
    app.add_handler(CommandHandler("html", cmd_html))
    app.add_handler(CommandHandler("callout", cmd_callout))
    app.add_handler(CommandHandler("todostyle", cmd_todostyle))
    app.add_handler(
        MessageHandler(filters.Document.ALL & filters.CaptionRegex(r"^/restore\b"), cmd_restore)
    )
    app.add_handler(MessageHandler(filters.COMMAND, cmd_unknown))
    app.add_handler(MessageHandler(~filters.COMMAND, handle_message))
    app.run_polling()


if __name__ == "__main__":
    main()
```

requirements.txt

```
python-telegram-bot>=21.0
requests>=2.31
tzdata
```

Dockerfile

```
FROM python:3.12-slim

RUN apt-get update \
    && apt-get install -y --no-install-recommends gnupg \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY bot.py .

CMD ["python", "bot.py"]
```

Version 1.2 installs `gnupg` in the image. The bot uses it to encrypt notes when you point it at a self-hosted server. Default Notesnook doesn't need it, but it does no harm.

fly.toml

Change `app` to a name of your own; it must be unique across all of Fly.io. `iad` is Virginia, so pick a region near you if you like (`fly platform regions` lists them).

```
# Change this to a globally unique name, e.g. "yourname-notesnook-bot"
app = "your-unique-app-name"
primary_region = "iad"

[build]

# No [http_service]: the bot polls Telegram, so it needs no public port.

[[vm]]
  size = "shared-cpu-1x"
  memory = "256mb"

# Keeps saved tags (/data/tags.json) across restarts and deploys.
# Create the volume once first: fly volumes create tagdata --region iad --size 1
[mounts]
  source = "tagdata"
  destination = "/data"
```

The `[mounts]` block attaches a small disk where the bot keeps your saved tags. It costs about 15 cents a month for the 1 GB minimum. If you don't want tags, delete the block and skip the volume step in Step 4. The bot still works, but tags saved with `/tag` are lost each time you redeploy. The volume's region must match `primary_region`.

.dockerignore

```
# Ignore everything, then allow only what the image needs.
*
!bot.py
!requirements.txt
```

The file name starts with a dot. On a Mac, press Cmd+Shift+. in Finder to show hidden files, and check that it wasn't saved as `dockerignore`. It stops Fly from uploading anything else in the folder.

Step 4: Deploy to Fly.io

Run every command below from inside your project folder.

1. **Install the Fly CLI.**
   - Windows (PowerShell): `iwr https://fly.io/install.ps1 -useb | iex`
   - Mac or Linux: `curl -L https://fly.io/install.sh | sh`
2. **Sign up or log in.** Run `fly auth signup` for a new account, or `fly auth login` if you already have one.
3. **Create the app.** Use the same name you put in `fly.toml`:

```
fly apps create your-unique-app-name
```

4. **Store your secrets.** They stay encrypted on Fly and never appear in your code. Use the three values from Steps 1 and 2:

   ```
   fly secrets set TELEGRAM_BOT_TOKEN="your-bot-token" NOTESNOOK_INBOX_KEY="your-inbox-key"
   ALLOWED_USER_ID="123456789"
   ```

   Optional: the bot titles list notes with the time, in New York time by default. To change the zone, add `BOT_TIMEZONE="Europe/London"` or any other tz database name.

   Optional: to stop the bot embedding photos and files, and save a placeholder line instead, add `EMBED_MEDIA="0"` the same way.

   Self-hosted only: also set your server address. Use the key from your self-hosted account for `NOTESNOOK_INBOX_KEY`. Run `fly secrets set NOTESNOOK_SERVER_URL="https://notesnook.example.com"`. For default Notesnook, don't set it. If you set it earlier and want to switch back, run `fly secrets unset NOTESNOOK_SERVER_URL`.

   Optional: to load some tags the first time the bot runs, add `NOTESNOOK_TAGS='{"groceries": "67aecf3b9e1398484554bc90"}'`. It is only read when the tag file doesn't exist yet. After that, use `/tag` in Telegram.

5. **Create the tag volume.** Do this once, before the first deploy, in the same region as `primary_region` in `fly.toml`:

   ```
   fly volumes create tagdata --region iad --size 1
   ```

6. **Deploy.**

   ```
   fly deploy
   ```

7. **Keep it to one machine.** Fly may start two by default, and two copies polling Telegram will fight each other.

   ```
   fly scale count 1
   ```

8. **Check that it's running.** `fly status` should show one machine in the `started` state, and `fly logs` should show no Python errors.

Fly may warn that nothing is listening on a port. You can ignore that, because the bot connects out to Telegram and doesn't serve a website. If it warns that your build context is huge, you're deploying from a crowded folder; move the five files into their own folder.

Step 5: Test it

Open your bot in Telegram and send it a message. A working setup replies with "✓ Saved to Notesnook" within a couple of seconds. The note then appears in Notesnook after its next sync: pull down to refresh on mobile, or click the sync icon on desktop.

Try these in order:

You send	You should get
`Hello world`	A note titled "Hello world"
A message starting with `- [ ] milk`	A checklist note titled like "Telegram note, Oct 15, 8:15 AM"
`/todostyle`	A reply saying the todo style is now simple. Send it again to switch back to header
`/callout warning Test` and then a line of text on the next line	A note titled "Test" containing a warning callout
`/html <h1>Hi</h1><p>Some <b>bold</b> text</p>`	A note titled "Hi" with the bold text formatted
A photo with the caption `Receipt`	A note titled "Receipt" with the photo embedded, and the reply "✓ Saved to Notesnook (media embedded)"

Then send this as one message:

```
Groceries:
- [ ] milk
- [x] eggs
```

You should get a note titled "Groceries" containing a checklist whose header also reads "Groceries", with milk unchecked and eggs checked. The header's counter should read 1/2. (Send `/todostyle header` first if you changed the style.)

Self-hosted: send `/ver` first and check that Destination shows your server address. Then send a test message. "✓ Saved to Notesnook" means your server accepted the encrypted note. Sync Notesnook while logged in to the self-hosted account to see it.

Tags (optional): send `/ver`; it should say `Saved tags: 0 (on a Fly volume)`. In Notesnook, right-click a tag and choose **Copy ID**, then send `/tag groceries <the id>`. The bot confirms and sends you a backup file. Then send a note whose last line is `#groceries`. The reply should end with "(tags: groceries)", and the note should carry that tag after a sync. The reply only means the bot found a saved ID and sent it. If the note arrives without the tag, the ID doesn't exist in the Notesnook account you sync with. Copy it again from that account.

Two last checks:

- Run `fly machine restart`, wait about 10 seconds, and send another message. The bot should answer without any extra steps.
- Ask a friend to message the bot. It should stay silent.
- Send `/help` and `/ver`. The bot should reply with the usage guide and its version number. The /ver reply also shows the destination: "hosted Notesnook inbox" for default Notesnook, or your server address if you are self-hosting. Send /setup to see the setup steps for both.

Writing notes

Formatting you apply in Telegram (bold, italic, underline, strikethrough, links, inline code, code blocks, quotes) carries over to the note. Typed markdown works for these line prefixes:

You type	Notesnook gets
`- item`, `* item` or `• item`	Bullet list
`1. item` or `1) item`	Numbered list
`- [ ] item`	Unchecked todo
`- [x] item`	Checked todo
`# Heading` up to `###### Heading`	Heading

Titles. The first line of your message becomes the note title. If the message starts with a list item, the title is generic instead, like "Telegram note, Oct 15, 8:15 AM", so the first item isn't repeated as the title.

Commands. Send `/help` to see a usage guide inside Telegram, including these markdown shortcuts. The commands are:

Command	What it does
`/help`	Shows the usage guide (it arrives in more than one message when it is long)
`/setup`	Shows how to connect the bot to default or self-hosted Notesnook
`/ver`	Shows the bot's version, the Fly.io region, the destination, the saved tag count, the time zone, whether media embedding is on and the todo style
`/html`	Saves the HTML that follows as a note, exactly as typed
`/callout`	Saves a callout box, in one message or by answering prompts
`/todostyle`	Switches todos between a header task list and a simple checklist
`/tag`, `/untag`, `/tags`, `/tagsfile`	Manage the saved tags (see Tags below)
`/cancel`	Stops a `/tag`, `/untag` or `/callout` prompt

Setup help. Send `/setup` for a short guide to connecting the bot to default or self-hosted Notesnook, including where to create the key and which secrets to set.

Tags. Put tags on the last line of a message, after your text, for example `#groceries #home`. The bot removes that line from the note and applies the matching Notesnook tags. A `#word` anywhere else is ordinary text, and a message that is only a tag is saved as a normal note.

Notesnook gives the bot no way to read your tag list, so you save each tag once with `/tag name id`. Copy the ID in Notesnook by right-clicking the tag and choosing **Copy ID**. `/untag name` forgets a tag in the bot only; it never deletes it in Notesnook. `/tags` lists what is saved. Send `/tag` or `/untag` on its own and the bot asks for what it needs; `/cancel` stops the prompt. The `#` suggestions Telegram shows while you type come from Telegram, which offers hashtags you have used before. They are not the bot's list. A tag the bot doesn't know stays in the note as text, and the reply says it wasn't found.

Tag backups. After every `/tag` and `/untag`, the bot sends you the tag file in Telegram. `/tagsfile` sends it on request. To restore, send the file back to the bot with the caption `/restore`. Fly also keeps daily snapshots of the volume for 5 days by default, but the copy in your chat is the one that survives losing the volume.

Checklist headers. Notesnook shows a header row above every checklist. Put a line directly above your todos, with no blank line between, and it becomes that header's title:

```
Groceries:
- [ ] milk
- [ ] eggs
```

With a blank line between, or no line above, the header reads "Untitled". The header's counter (for example 1/2) counts every item in the list, nested or not.

Todo style. Todos can be saved in one of two styles, and `/todostyle` switches between them. The choice is remembered.

- **header** (the default): Notesnook's task list, with a title bar, a progress bar and the checked/total count. The line above the todos becomes the title.
- **simple**: a plain checklist with no header or count. The line above the todos stays in the note as normal text.

Send `/todostyle` to flip between them, or `/todostyle header` or `/todostyle simple` to set one directly. The style applies to todos in normal messages and in `/callout` bodies. It does not change `/html`. The setting is stored in the tag file on the Fly volume, so it survives redeploys. The tag backup file does not include it, so set the style again after a `/restore`.

Callouts. `/callout` saves a Notesnook callout box. Start a message with `/callout`, then an optional type and a title, then the body on the next lines:

```
/callout warning Back up first
Export your notes before upgrading.
```

The types are note, abstract, summary, tldr, info, todo, tip, hint, important, success, check, done, question, help, faq, warning, warn, caution, attention, failure, fail, missing, danger, error, bug, example, quote and cite. With no type, or an unknown one, the callout is `info`. With no title, the type name is used. The body can use lists, todos and Telegram formatting, and the note title is the callout title.

Send `/callout` on its own and the bot asks for the type, the title and the text, one message each. Send `skip` to leave a step out, or `/cancel` to stop. An unanswered prompt expires after 5 minutes.

Tags work in a callout: put saved tags on a line of their own at the end, or at the end of the last line. Because the first word of the title is read as the type when it matches one, a title like "Tip of the day" needs a type in front: `/callout info Tip of the day`.

Raw HTML. A normal message escapes any tags you type, so `<b>` appears as plain text. To save real HTML, start the message with `/html`:

```
/html <h1>Groceries</h1><ul class="checklist"><li class="checklist--item">milk</li><li class="checklist--
item checked">eggs</li></ul>
```

The HTML is sent as typed. The note title is the first heading (`<h1>` to `<h6>`), or the date and time if there is none. Every tag must be closed: the bot refuses to save HTML with an unclosed or mismatched tag, because Notesnook would turn the whole note into a code block. A saved `#tag` on the last line works as usual. Notesnook cleans the HTML on your device after it arrives, so unsafe parts such as scripts are removed.

Notesnook only recognises some structures by their class names. Checklist items need `class="checklist--item"` (plus `checked` when done) inside `<ul class="checklist">`, and a callout is `<div class="callout" data-callout-type="info">` with a heading as its first child. A simple checklist uses `<ul class="simple-checklist">` and `<li class="simple-checklist--item">`.

Photos and files. The Inbox API has no attachment field, so the bot embeds media in the note's HTML, the way the Notesnook email worker does. Photos and image files (JPEG, PNG, GIF, WebP) become embedded images, and text files (.txt, .md, .csv, .json, code and similar) become code blocks. Common types get syntax highlighting from their extension: .py, .js, .ts, .json, .yaml and .yml, .sh, .sql, .md and .markdown, .css, .toml, .ini, .csv, and .html and .xml. Other types stay plain text. Each file can be up to 5 MB, and a whole note up to 8 MB. Videos, voice notes and other file types get a placeholder line such as "[Video attached in Telegram, not embedded]" plus the caption, if any. Each photo in an album becomes its own note. Embedded images are part of the note, not Notesnook attachments, and Notesnook's documentation suggests it may strip embedded images, but in testing they display correctly. If they ever stop appearing, run `fly secrets set EMBED_MEDIA=0` to switch embedding off, and use your phone's share sheet for real attachments.

Limits. Lists don't nest, and numbering always starts at 1. A message with a code block, a quote, or bold text that runs across several lines skips the list conversion and is saved as simple text, so nothing is lost.

Troubleshooting

Start with `fly logs`. It shows startup errors and every request the bot handles.

What you see	Likely cause	Fix
The bot never replies	Wrong user ID, bot not running, or two machines running	Check your ID with @userinfobot, then run `fly status` and make sure exactly one machine is started
The app crashes at startup with a `KeyError`	A secret is missing	Run `fly secrets list`; all three secrets must appear
Reply says "Failed (401)"	The Inbox key is wrong, expired or revoked	Create a new key, then run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`
Reply says "Failed (403)"	The Inbox API is turned off	Turn on Enable Inbox API in Notesnook settings
Reply says "Failed (400)"	Notesnook rejected the note; the reply names the field	Check that `bot.py` matches this guide
Reply says "Failed (429)"	Rate limit of 60 requests a minute	Wait a minute and resend
Reply says "Saved" but no note appears	Notesnook hasn't synced yet	Force a sync, then check Failed inbox items under Settings > Account > Inbox
A note arrives as a block of raw markup	The HTML had an unclosed tag	Not expected with this bot; note the exact message you sent
Deploy is slow and warns about a huge build context	Deploying from a crowded folder	Use a dedicated folder with the `.dockerignore` file
A photo note arrives without the picture	The photo was over 5 MB, or Notesnook changed how it handles embedded images	Read the bot's reply for "not embedded". If images never show, run fly secrets set EMBED_MEDIA=0

Self-hosted server

What you see	Likely cause	Fix
Reply says "Failed (401)"	The key is from a different account, for example the default Notesnook account	Create a new key on your self-hosted account and run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`

What you see	Likely cause	Fix
Reply says "Failed (404)"	The server address is wrong, or it points at an auth, events, monograph or attachments address	Set `NOTESNOOK_SERVER_URL` to the main sync address with no slash at the end
Reply says "Network error"	Your server is down or unreachable from Fly.io	Open the server address in a browser, then check that the server is running
`/ver` still shows "hosted Notesnook inbox"	`NOTESNOOK_SERVER_URL` is not set	Run `fly secrets set NOTESNOOK_SERVER_URL="https://notesnook.example.com"`
Reply says "Saved" but no note appears	Notesnook is logged in to a different account, or has not synced	Log in to the self-hosted account and force a sync
Logs mention `gpg failed`	`gnupg` is missing from the image	Use the Dockerfile in this guide and run `fly deploy` again

Tags

What you see	Likely cause	Fix
Reply says "Tag not found"	The tag was never saved in the bot	Send `/tag name id` with the ID copied from Notesnook
The `#tag` line stays in the note	The tag isn't saved yet, or the line isn't the last line	Save the tag, and put the tags alone on the last line
Reply says "(tags: name)" but the note has no tag	Notesnook silently skips a tag ID that doesn't exist in the account it syncs with. IDs are per account, so one saved from default Notesnook won't exist on a self-hosted server	Copy the ID again from the account the bot sends to, send `/tag name id`, and send a new note. Notes already saved need the tag added by hand
`/callout` tag not applied	The tag isn't saved in the bot, or it isn't at the end of the message	Save it with `/tag`, and put it on its own last line or at the end of the last line
Typing `#` shows tags you never saved	Telegram suggests hashtags you have used before	Nothing to fix. `/tags` lists what the bot knows
`/ver` says "NOT persistent: no volume mounted"	The volume isn't attached	Check the `[mounts]` block in `fly.toml`, then run `fly deploy`
Tags vanish after a deploy	The same: no volume	Create the volume, redeploy, then send the backup file back with `/restore`

Commands and formatting

What you see	Likely cause	Fix
The checklist header shows 0/2 although an item is checked	The note was saved by a bot older than 1.4.0, or by HTML without the `checklist--item` class	Update to 1.4.0 and send the note again. Notes already saved keep the old count. With `/html`, give each item `class="checklist--item"`
`/html` replies "Not saved: the HTML has an unclosed or mismatched tag"	A tag isn't closed, or tags are closed in the wrong order	Fix the tags and send it again
Your `<b>` or `<ul>` shows as plain text in the note	It was sent as a normal message, which escapes tags	Start the message with `/html`
A `/callout` came out as `info`, or the first word of the title vanished	The first word was read as a type, or wasn't a known type	Put a type first, for example `/callout info Tip of the day`
A todo list has no header	The todo style is `simple`	Send `/todostyle header`
The todo style went back to `header` after a deploy	No Fly volume is mounted, so the setting wasn't kept	Create the volume (Step 4), redeploy, and set the style again
A `.py` file shows as plain text	Only the listed file types are highlighted	Nothing to fix; see Photos and files

Update the bot. Edit `bot.py` and run `fly deploy` again.

Replace a key. Turning the Inbox API off revokes every key. After turning it back on, create a new key and run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`. Setting a secret restarts the bot.

Replace a key (self-hosted). Create the new key in the Notesnook app that is logged in to your self-hosted account, then run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`. If you reset your account's encryption keys, create a new Inbox key as well. The bot fetches the public key again on its own.

Stop or remove it. `fly scale count 0` pauses the bot. `fly apps destroy your-unique-app-name` deletes it.

Limitations and security

- **Attachments are embedded, not attached.** Photos and text files are embedded in the note. Videos, voice notes and other files get a placeholder line.
- **`/html` is not escaped.** The bot sends your HTML as typed. Notesnook cleans it on your device, but only you can use the bot, so the content is yours.
- **Beta API.** The Notesnook Inbox API may change. If the bot breaks after a Notesnook update, check the official guide.

- **One user per bot.** The bot answers a single Telegram user ID. Anyone who wants their own bridge should deploy their own copy with their own bot token and Inbox key.
- **Protect your secrets.** Never put the bot token or Inbox key in `bot.py`, a Git repository or a screenshot. If either leaks, send `/revoke` to @BotFather for the bot token, and delete the key under Settings > Inbox in Notesnook.
- **Privacy.** Telegram bot chats are not end-to-end encrypted, so Telegram can read your messages. Notesnook's hosted inbox server receives each note over HTTPS, encrypts it with your public key and forwards it; its documentation says it stores nothing. If you'd rather not send notes through it, run your own Notesnook sync server and set `NOTESNOOK_SERVER_URL`. The bot then encrypts each note on Fly.io, so your server receives only the encrypted note. Telegram and Fly.io still see the message text. Notesnook also documents self-hosting the inbox server.